HiST: Hierarchical Semantic Tree Augmentation for Generative Recommendation

Bocheng Pan
Institute of Microelectronics, Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of Sciences
Beijing, China
panbocheng@ime.ac.cn

Hailong Shi*
Institute of Microelectronics, Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of Sciences
Beijing, China
shihailong2010@gmail.com

Xingyu Gao*
Institute of Microelectronics, Chinese Academy of Sciences
Beijing, China
University of Chinese Academy of Sciences
Beijing, China
gxy9910@gmail.com

Abstract

Recommendation systems are indispensable for helping users navigate the vast product catalogs of modern online platforms. Recently, generative recommendation has emerged as a promising paradigm due to its unified and flexible formulation, and it has been adopted in various commercial scenarios. However, we find that different tokenization algorithms share a common issue: semantic branch drift, where semantically similar items are often assigned to different branches of the semantic tree. This phenomenon leads to severe information loss in the generated semantic IDs and significantly degrades recommendation accuracy. To address this problem, we propose HiST (Hierarchical Semantic Tree Augmentation), a novel framework that leverages branch information to bridge the semantic gap. The core innovation of HiST lies in using a trained tokenizer to construct a semantic tree over all items and extracting hierarchical information through a bottom-up approach. This hierarchical information is then fused with the original semantic IDs to enhance representation capacity. During end-to-end training, HiST improves recommendation performance by introducing separate prediction objectives for semantic IDs and hierarchical description components, enabling the model to effectively learn both discrete structure and semantic relationships. Experimental results demonstrate that HiST significantly improves recommendation performance across multiple datasets. Furthermore, experiments verify that HiST can be effectively integrated with various existing tokenization methods to enhance recommendation model performance, achieving consistent improvements across evaluated settings.

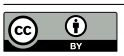
CCS Concepts

• **Information systems** → **Recommender systems**; *Language models*; • **Computing methodologies** → *Natural language processing*; *Machine learning*.

Keywords

Generative Recommendation; Semantic Tree; Large Language Model

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817719

ACM Reference Format:
Bocheng Pan, Hailong Shi, and Xingyu Gao. 2026. HiST: Hierarchical Semantic Tree Augmentation for Generative Recommendation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817719

1 Introduction

In recent years, generative recommendation systems have gained significant attention due to their outstanding performance in sequence modeling and next-item prediction [12, 28]. These systems transform recommendation tasks into sequence generation problems, learning user behavior patterns through autoregressive generative model architectures. The success of generative recommendation heavily relies on effective tokenization strategies that convert items into discrete tokens suitable for sequence modeling [19]. Advanced tokenizers, such as Residual Quantized Variational Autoencoder (RQVAE), employ hierarchical multi-codebook structures to achieve high-quality discrete representations [4, 27], significantly expanding the representation capacity through fine-grained semantic encoding. This hierarchical approach transforms unrelated item IDs into interconnected semantic ID encodings that capture rich semantic relationships, providing strong technical support for generative recommendation systems [10, 24].

However, our analysis indicates that existing tokenization methods face a critical issue that we term semantic branch drift. As illustrated in Figure 1, during the encoding phase, the tokenizer's optimization process overemphasizes reconstruction and generation objectives, causing semantically similar items to be quantized into different branches and disrupting the hierarchical semantic structure of discrete tokens. This semantic drift phenomenon significantly increases the difficulty for recommendation models to understand and predict semantic IDs, making the hierarchical and local structures of the generated discrete tokens less amenable to downstream models and limiting recommendation effectiveness.

To address the above problems, we propose HiST (Hierarchical Semantic Tree Augmentation), which supplements hierarchical semantic information to bridge the semantic gap caused by branch drift. The core innovation of HiST lies in leveraging the trained tokenizer to construct a complete semantic tree and extracting hierarchical information through a bottom-up approach. This hierarchical information is then fused with the original semantic IDs to enhance representation capabilities. During end-to-end training, HiST optimizes recommendation performance through separate

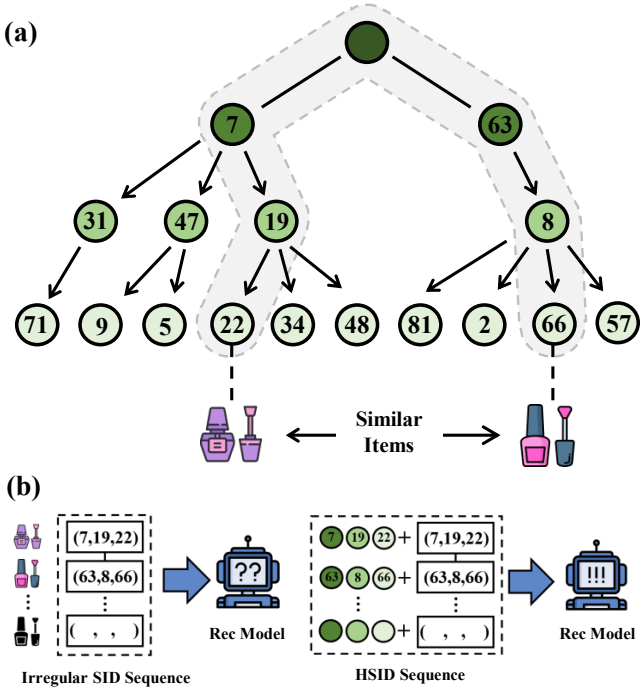


Figure 1: Illustration of the Semantic Branch Drift problem and the HiST solution. (a) The tokenizer's semantic branch drift challenges recommendation models by mapping semantically similar items to different branches. (b) HiST alleviates this problem by injecting hierarchical semantic information from the semantic ID tree into the recommendation model, improving understanding and prediction performance.

prediction losses for both semantic IDs and hierarchical information components, enabling models to better understand the hierarchical semantic structure. This approach achieves significant performance improvements across multiple datasets and shows good empirical generalization across state-of-the-art generative recommendation models and tokenization methods evaluated in this work.

The main contributions of this paper are summarized as follows:

- We provide a systematic empirical analysis of the semantic branch drift phenomenon in existing tokenization methods, where semantically similar items are assigned to different branches of the semantic tree, significantly increasing the difficulty for recommendation models to comprehend ID encodings and affecting recommendation accuracy.
- We propose HiST, a novel tokenization enhancement method that supplements hierarchical semantic information to bridge the semantic gap caused by branch drift. HiST leverages bottom-up layer information refinement to extract hierarchical descriptions that are fused with original semantic IDs for enhanced representation, achieving consistent gains with multiple tokenization methods and state-of-the-art generative recommendation models.
- We conduct extensive experiments on multiple benchmark datasets to validate the effectiveness of the HiST method, with experimental results showing that our approach achieves significant performance improvements across all datasets and tokenization approaches.

2 Related Work

2.1 Sequential Recommendation Models

Sequential recommendation systems leverage user interaction sequences to learn representations for users and items, capturing both long-term preferences and short-term dynamics. Early models such as GRU4Rec utilized Recurrent Neural Networks to capture temporal dependencies in user behavior [7], while Caser applied Convolutional Neural Networks to treat sequences as "images" for spatial feature extraction [23]. The advent of Transformer-based models has significantly advanced this field, with SASRec employing self-attention mechanisms for autoregressive next-item prediction [11] and BERT4Rec introducing bidirectional context modeling inspired by masked language modeling [22]. More advanced models like PinnerFormer and FDSA have further enhanced user representation by leveraging Transformers for long-term behavior modeling and feature integration [17, 30]. Recent works including ZESRec, UniSRec, and RecFormer have emphasized cross-domain transferability by incorporating textual features and employing contrastive or language-oriented learning techniques, with RecFormer unifying language understanding and sequence recommendation through bidirectional Transformers [2, 9, 14].

2.2 Generative Recommendation Systems

The increasing popularity of generative models across various domains has led to a paradigm shift in recommendation systems from discriminative to generative approaches. Unlike traditional methods that learn item embeddings and rely on external Approximate Nearest Neighbor (ANN) indices to retrieve candidates by scoring relevance [3], generative models directly generate item identifiers rather than computing ranking scores for each item. P5 transformed various recommendation tasks into natural language sequences, providing a universal framework for recommendation completion [5]. TIGER pioneered the application of generative retrieval to recommendations by using residual/vector-quantized autoencoders to create semantically rich indexing identifiers, which are then utilized by transformer-based models to generate item identifiers from user histories [19].

Building upon this foundation, several approaches have emerged to address different aspects of generative recommendation. LC-Rec enhanced TIGER's approach by aligning semantic identifiers with collaborative filtering techniques through additional alignment tasks [31]. SEATER focused on maintaining semantic consistency through balanced k-ary tree-structured indexes refined by contrastive and multi-task learning [21]. EAGER proposed a dual-stream generation framework achieving information fusion through parallel prediction of semantic encoding and collaborative/behavioral encoding [25], though architectural complexity and insufficient interaction during decoding can hinder deep fusion. COBRA attempts to combine discrete semantic encoding with additional dense embeddings on top of semantic IDs through joint/cascaded training, enabling embeddings to carry both collaborative and semantic information simultaneously [27], though this

design largely keeps semantic IDs and dense features as parallel channels instead of explicitly regularizing the hierarchical structure of the semantic ID (SID) space itself, limiting its ability to directly correct semantic branch drift. Systematization and tooling have also emerged: GRID provides modular ablations and a unified framework for Generative Recommendation with SIDs [10], and UNGER fuses semantic and collaborative signals into unified generative codes [26].

2.3 Semantic ID Enhancement

Numerous works have focused on improving tokenizers and enhancing semantic IDs to address various limitations in generative recommendation systems. Beyond the systematization efforts mentioned above, GenSAR designs dual-purpose identifiers for unified retrieval and recommendation, targeting cross-task unified SID spaces [20]. RPG addresses efficiency concerns by transforming ordered short SIDs into unordered, parallel-predictable long SIDs, using multi-token parallel loss for training and significantly reducing autoregressive dependencies [8]. Other enhancement approaches include methods that focus on collision handling, uniqueness preservation, and alignment with industrial requirements for more robust semantic ID construction.

Despite these advances in tokenizer design and semantic ID construction, all these methods share a common limitation: semantic branch drift. Our work specifically targets this issue by proposing hierarchical semantic tree augmentation that explicitly leverages branch-level information to improve recommendation performance.

3 Analysis of Semantic Branch Drift

Modern tokenizers for generative recommendation models typically employ hierarchical quantization through multi-layer codebooks $\{C_1, C_2, C_3\}$ to tokenize item embeddings. Each item x is encoded into a multi-dimensional discrete semantic identifier:

$$z(x) = (z_1, z_2, z_3), \quad z_d \in \{1, \ldots, |C_d|\}, \quad d = 1, 2, 3 \tag{1}$$

This approach not only expands the representation capacity of encoded items but also aims to implicitly organize items in a hierarchical manner: first partitioned by layer-1 IDs z_1 forming coarse-grained categories, then subdivided by layer-2 IDs z_2 creating moderate-grained subclasses, and finally grouped by layer-3 IDs z_3 representing fine-grained units.

However, the expected hierarchical structure does not emerge in practical applications. Instead, tokenizers often assign similar items to different branches of the semantic tree, which disrupts the hierarchy and hurts recommendation performance. We use the term Semantic Branch Drift to describe the resulting mismatch between item-level semantic similarity and the branch assignments produced by the tokenizer: items that are semantically close in attributes or encoder space can end up scattered across unrelated branches of the semantic tree.

In this section, we analyze how semantic branch drift manifests in tokenizers and affects item sequences. We conduct experiments on the Amazon Beauty dataset with two well-trained tokenizers, RQVAE and R-KMeans, and use categories and brands as semantic references.

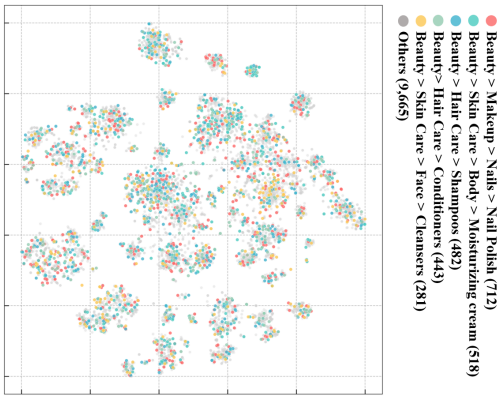


Figure 2: t-SNE visualization of the top-5 categories.

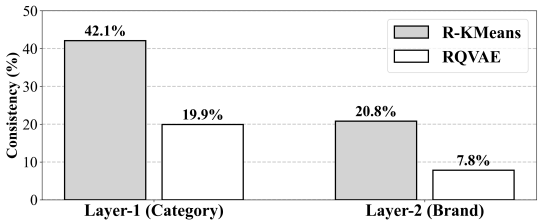


Figure 3: Semantic consistency comparison.

Categories Scatter Across Multiple Clusters. To intuitively demonstrate this phenomenon, we visualize the encoder output of RQVAE with t-SNE. Coloring points by the top-5 categories in Figure 2 shows that most clusters contain multiple categories and that items from the same category are scattered across many clusters. Even visually coherent categories that should ideally form compact regions instead appear as many small, interleaved islands spread over the map. In other words, items that share semantic labels do not occupy contiguous regions in the learned space. This fragmentation breaks the desired hierarchical coherence and indicates a misalignment between semantic structure and the tokenizer's learned representation space.

Same Branches Contain Multiple Categories. To quantify this effect across tokenizers, we measure attribute consistency within each semantic ID cluster. For each ID, we compute the share of the dominant category or brand and then average over all IDs. As shown in Figure 3, we observe many branches where one category is only moderately dominant and a long tail of items from other categories still share the same ID. Both RQVAE and R-KMeans exhibit notable mixing of different categories within the same branches, with RQVAE being clearly more severe. These low consistency scores indicate that many branches contain items from multiple semantic groups, and that semantic branch drift is a pervasive issue across current tokenization approaches.

Lower Semantic Drift Enables Better Sequence Continuity. Sequence-level case studies further highlight the impact of branch drift. As shown in Figure 4, R-KMeans produces ID sequences with clear and repeated patterns, making user behavior trajectories easier

Items ID Sequence	RQVAE SID Sequence	R-Kmeans SID Sequence	Item Message
[459]	[8_95_132]	[56_85_185]	[459] Title: Avalon Biotin B-Complex Thickening Conditioner, 14 Ounce; Brand: Avalon Organics; Categories: ['Beauty', 'Hair Care', 'Conditioners']; Price: 9.49;
[624]	[47_201_63]	[2_71_207]	[624] Title: DS Laboratories Revita Hair Stimulating Shampoo, 290-gram Bottle; Brand: DS Laboratories; Categories: ['Beauty', 'Hair Care', 'Shampoos']; Price: 11.36;
[625]	[110_4_189]	[125_103_244]	[625] Title: Lipogaine for Men: Intensive Treatment & Brand: Unknown; Categories: ['Beauty', 'Hair Care', 'Hair Loss Products', 'Hair Regrowth Treatments']; Price: 25.0;
[626]	[22_150_77]	[112_214_248]	[626] Title: Devoted 2 Black Supre Advanced Bronzer Tanning Lotion; Brand: Devoted Creations; Categories: ['Beauty', 'Skin Care', 'Sun', 'Self-Tanners & Bronzers']; Price: 17.48;
[607]	[39_244_101]	[125_211_148]	[607] Title: Kirkland Minoxidil 5% Extra Strength Hair Regrowth Brand: Categories: ['Beauty', 'Hair Care', 'Hair Loss Products', 'Hair Regrowth Treatments']; Price: 26.24;
[627]	[56_173_29]	[125_172_243]	[627] Title: Hairomega 3-in-1 Dht-blocking, Nutrient Providing, Brand: Unknown; Categories: ['Beauty', 'Hair Care', 'Hair Loss Products', 'Hair Regrowth Treatments']; Price: 22.71;
[628]	[143_66_210]	[2_242_126]	[628] Title: Nizoral AntiDandruff Shampoo, 7-Ounce Bottles; Brand: Nizoral; Categories: ['Beauty', 'Hair Care', 'Shampoos']; Price: 17.76;

Figure 4: Case study of user sequences illustrating semantic ID interpretability differences.

to interpret. In a typical hair-care sequence, treatments consistently share one small set of IDs and shampoos share another, so similar products appear as short, recurring ID subsequences. These stable local patterns allow recommendation models to quickly attach consistent semantic meaning to frequently appearing IDs. Under RQVAE, the same sequence is mapped to many distinct IDs that rarely repeat, and adjacent items with similar attributes often receive unrelated codes. As a result, RQVAE yields ID sequences that look much more random, even for semantically coherent purchase histories, which hampers the model's ability to learn stable item meanings and maintain semantic continuity along the sequence.

Based on the above analysis, we conclude that semantic branch drift explicitly impairs recommendation models' understanding of semantic ID encodings and leads to fragmented, hard-to-interpret sequences. Therefore, we need a method that exploits information from multiple levels of the tree to supplement the hierarchical semantics lost during tokenization. While R-KMeans exhibits lower semantic branch drift and produces more interpretable ID sequences, we mainly treat it as a diagnostic baseline rather than a replacement for residual quantization. In industrial generative retrieval systems, residual-quantized VAEs such as RQVAE remain the de facto choice for semantic ID tokenization due to their higher representational capacity and widespread adoption. HiST is therefore designed to remedy the structural weakness of RQVAE exposed by the above analysis, rather than replacing the underlying tokenizer.

4 Methodology

To address the Semantic Branch Drift problem in tokenization methods, we propose HiST (Hierarchical Semantic Tree Augmentation), which supplements hierarchical semantic information for enhanced item representation. The core innovation of HiST lies in leveraging any trained tokenizer to construct a complete semantic tree and extracting hierarchical descriptions through bottom-up layer information refinement. These descriptions are then fused with the original semantic IDs to provide enriched representations that help models comprehend connections between different branches. Unlike LLM-augmented recommenders that simply attach dense textual features alongside item embeddings or semantic IDs, HiST uses LLMs to reshape and strengthen the hierarchical structure of the SID space itself through the semantic tree and HSID representation. Figure 5 illustrates the overall framework of HiST.

4.1 Semantic Tree Construction

Prior analysis of residual quantization has revealed an "hourglass" phenomenon: when the hierarchy is too deep, only the first and last codebooks tend to be sufficiently utilized while intermediate layers are underused, compressing or even discarding information and preventing the IDs from forming ideal hierarchical semantics, whereas too shallow hierarchies lack representational capacity [13]. To avoid this hourglass behavior in practice, industrial generative retrieval systems typically adopt three layers of codebooks with around 256 codewords per layer, which we also follow in our design. Starting from any trained tokenizer, we encode all items in the dataset offline to construct a hierarchical semantic tree through a top-down incremental approach. For each item x in the catalog, the tokenizer produces a multi-dimensional semantic identifier $z(x) = (z_1, z_2, z_3)$ where $z_1 \in \{1, \ldots, |C_1|\}$, $z_2 \in \{1, \ldots, |C_2|\}$, and $z_3 \in \{1, \ldots, |C_3|\}$ represent the SID at each layer.

The hierarchical tree construction follows a systematic top-down incremental process to establish a layered structure that facilitates subsequent layer-wise information extraction. Items are first partitioned by their layer-1 ID z_1 to form the coarsest semantic categories, then incrementally refined by layer-2 ID z_2 to create moderate-grained subcategories, and finally organized by layer-3 ID z_3 to establish fine-grained semantic units. This incremental hierarchical organization ensures that each layer captures semantic information at different granularity levels, providing a structured foundation for extracting layer-wise descriptions that will be integrated with semantic IDs to create enhanced representations.

In our main instantiation, we thus build the semantic tree on top of a three-layer RQVAE tokenizer. Compared with R-KMeans, RQVAE offers a higher-capacity semantic code space but suffers from more severe semantic branch drift; HiST can be viewed as borrowing the desirable clustering behavior of K-means and reintroducing hierarchical structure on top of residual quantization via LLM-based descriptions and semantic clustering, while preserving the flexibility and capacity of RQVAE.

4.2 Hierarchical Information Extraction

Based on the semantic IDs, we construct a three-layer description tree where each upper layer contains refined information from the layer below. We employ carefully designed LLM prompts to systematically extract hierarchical semantic information through a bottom-up approach, progressively abstracting from individual product descriptions to broader semantic themes. The prompts ensure that each extracted description captures commonalities while maintaining distinctiveness between different semantic branches. Detailed prompt designs can be found in Appendix C. This bottom-up extraction strategy enables effective semantic abstraction from fine-grained product characteristics to coarse-grained categorical themes.

Layer 3. Items sharing the same complete semantic identifier (z_1, z_2, z_3) are naturally grouped at the finest granularity level. For these layer-3 nodes, we directly utilize the original item descriptions

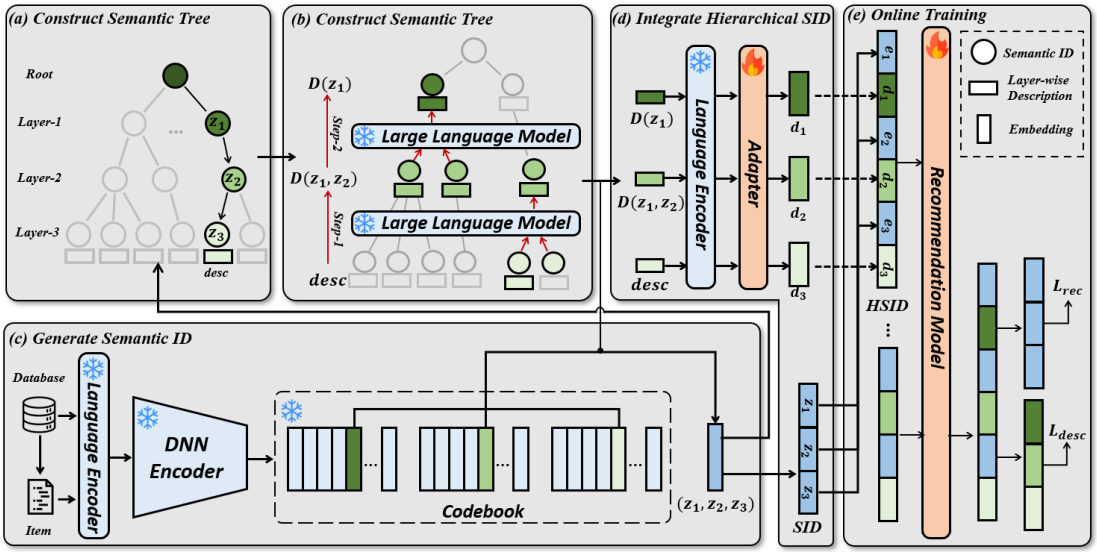


Figure 5: Overall framework of HiST. The method constructs a hierarchical semantic tree from RQVAE-generated semantic IDs, employs bottom-up layer information refinement to extract hierarchical descriptions, and integrates them with semantic IDs to enhance recommendation.

as the semantic foundation, denoted as $desc(z_1, z_2, z_3)$, since they contain rich semantic information capturing specific item characteristics. Each layer-3 node typically contains highly similar items that share the same quantized representation and similar functional properties.

Layer 2. For layer-2 nodes corresponding to (z_1, z_2) combinations, we systematically analyze multiple layer-3 product descriptions that share the same semantic branch. We employ specialized prompts to guide the large language model in extracting common semantic features, focusing on recurring patterns in product categories, brand characteristics, and functional attributes:

$$D(z_1, z_2) = \text{LLM}(\{desc(z_1, z_2, z_3) | \forall z_3 \in \text{children}(z_1, z_2)\}) \quad (2)$$

Layer 1. At the layer-1 level, we aggregate multiple layer-2 semantic summaries to identify fundamental semantic concepts. This requires higher-level abstraction prompts to discover overarching themes and categorical distinctions that unite different layer-2 branches:

$$D(z_1) = \text{LLM}(\{D(z_1, z_2) | \forall z_2 \in \text{children}(z_1)\}) \quad (3)$$

where $\text{LLM}(\cdot)$ represents the large language model's extraction capabilities using carefully designed hierarchical prompts that ensure semantic coherence and distinctiveness across different branches.

4.3 Hierarchical Semantic ID Integration

To integrate the extracted hierarchical descriptions with original semantic IDs, we employ a three-stage process: description encoding, dimension alignment, and information fusion. This process creates enhanced representations that preserve discrete tokenization benefits while augmenting with rich hierarchical semantic context.

Description Encoding. We encode all extracted hierarchical descriptions using the same pre-trained language model used for tokenizer training to maintain semantic coherence. This transformer-based model captures both syntactic and semantic relationships, transforming textual descriptions into high-dimensional vector representations suitable for integration with SIDs.

Dimension Alignment. A critical challenge lies in aligning description embedding dimensions with semantic ID embeddings, as they originate from different training processes. We employ adapter networks to map description embeddings to the semantic ID embedding space:

$$d = \mathcal{A}(e_{\text{desc}}) = \mathcal{A}(\text{LM}(\text{description})) \quad (4)$$

where $\mathcal{A}(\cdot)$ represents the adapter function consisting of lightweight feed-forward layers with non-linear activation and dropout for stable training.

Information Fusion. We integrate dimension-aligned description embeddings with corresponding semantic ID embeddings to create enhanced representations. The original semantic IDs (z_1, z_2, z_3) are embedded to obtain e_1, e_2, e_3, while extracted descriptions are processed to obtain d_1, d_2, d_3. We denote the hierarchical semantic ID (HSID) as the enhanced representation obtained by supplementing the original semantic ID with hierarchical information:

$$HSID = [e_1, d_1, e_2, d_2, e_3, d_3] \quad (5)$$

This integration strategy enables the model to leverage complementary information from both components. The semantic ID embeddings e_i provide discrete categorical information essential for efficient tokenization and sequence modeling, while the description embeddings d_i offer rich contextual semantic information that helps disambiguate similar items across different branches. The

interleaved concatenation of e_i and d_i pairs ensures that semantic and descriptive information are tightly coupled at each hierarchical level, allowing the model to simultaneously consider both the discrete structure and semantic nuances. This representation effectively bridges the gap caused by semantic branch drift by enabling recommendation models to better understand hierarchical semantic structures.

4.4 End-to-End Training

The recommendation model training follows a sequential item prediction paradigm using a Transformer encoder-decoder architecture, well-suited for capturing temporal dependencies and sequential patterns in user behavior. During training, sequences of user interactions are processed where each item is represented by its HSID as defined in Equation (4). The Transformer processes these dual-component representations to predict the next item, leveraging both structural information from semantic IDs and contextual information from hierarchical descriptions. However, for the prediction target, we use the original semantic IDs rather than the full HSID representation, as semantic IDs provide more efficient and precise discrete targets for sequence generation. The training process jointly optimizes HSID representations and sequential prediction models through a unified reconstruction loss function.

To optimize model performance, HiST introduces separate prediction losses for both SID and description components. The model learns to predict the next item by optimizing the following combined loss function:

$$\mathcal{L} = \mathcal{L}_{\text{SID}} + \lambda \mathcal{L}_{\text{desc}} \tag{6}$$

The semantic ID prediction loss $\mathcal{L}_{\text{SID}}$ follows the standard cross-entropy formulation for discrete sequence modeling:

$$\mathcal{L}_{\text{SID}} = -\sum_{u} \sum_{t=1}^{|S_u|} \log P(z_t^{(u)} | HSID_{<t}^{(u)}; \theta, \Phi) \tag{7}$$

where u indexes users, S_u represents the interaction sequence of user u, $z_t^{(u)}$ is the semantic ID of the item at position t, $HSID_{<t}^{(u)}$ denotes all previous items represented as HSIDs in the sequence, θ represents the Transformer model parameters, and Φ represents the HSID embedding parameters including adapter weights.

The description prediction loss $\mathcal{L}_{\text{desc}}$ employs contrastive learning to refine the description vectors, enabling them to discern between similar and dissimilar items:

$$\mathcal{L}_{\text{desc}} = -\sum_{u} \sum_{t=1}^{|S_u|} \log \frac{\exp(\cos(\hat{d}_t^{(u)}, d_t^{(u)}))}{\sum_{j \in \text{Batch}} \exp(\cos(\hat{d}_t^{(u)}, d_j))} \tag{8}$$

where $\hat{d}_t^{(u)}$ is the predicted description vector, $d_t^{(u)}$ is the ground truth description vector for the positive item, and d_j represents the description vectors of items within the batch. The cosine similarity $\cos(\hat{d}_t^{(u)}, d_t^{(u)})$ measures the alignment between predicted and ground truth description vectors, while the contrastive term ensures that the predicted description is more similar to the ground truth than to other items in the batch.

This dual-component loss design enables the model to leverage the rich hierarchical semantic information from HSID representations during encoding while maintaining efficient semantic ID prediction targets. The HSID serves as auxiliary information that helps the model better understand the semantic relationships and hierarchical structure, while the semantic ID prediction target ensures precise discrete token generation suitable for generative modeling. The contrastive description loss further enhances the model's ability to capture fine-grained semantic distinctions, ultimately bridging the semantic branch drift problem in recommendation tasks.

Importantly, the large language model used for hierarchical description extraction is invoked only in the offline semantic tree construction stage. During online inference, HiST employs the same Transformer-based generative recommender as TIGER given HSID embeddings, so the serving-time latency and computational cost remain comparable to standard SID-based generative recommendation systems.

5 Experiments

5.1 Experimental Setup

Datasets. We conduct experiments on three widely used datasets from the Amazon Product Reviews dataset: Beauty, Toys, and Sports. Table 2 summarizes the detailed statistics of these datasets. Following previous work, we apply 5-core filtering [16], which removes users and items with fewer than five interactions to ensure data quality. To standardize training, each user's interaction history is truncated or padded to a fixed length of 20, retaining the most recent interactions (a common practice in sequential recommendation [11, 22]). For data splitting, we adopt the widely used leave-one-out strategy: for each user, the most recent interaction is held out for testing, the second most recent for validation, and the remaining interactions are used for training.

Baselines. We conduct a comprehensive comparison of HiST against a wide spectrum of state-of-the-art models, which we group into three distinct categories. *(1) Traditional Sequential Models:* LightGCN [6], Caser [23], and GRU4Rec [7]; *(2) Transformer-based Models:* P5 [5], BERT4Rec [22], and SASRec [11]; *(3) Generative Recommendation Models:* TIGER [19], LC-Rec [31], and COBRA [27]. See Appendix B for more details on baselines.

Evaluation Protocol. We adopt Recall@K and NDCG@K with $K \in \{5, 10\}$ for main comparisons and $K = 5$ for ablation studies (to better capture hierarchical semantic effects). We perform full ranking over the entire item catalog to eliminate sampling bias. For generative methods, we set beam size to 20 during inference to balance diversity and computational efficiency.

5.2 Implementation Details

Hardware and Infrastructure. All experiments are conducted on 8 NVIDIA RTX 5090 GPUs with sufficient memory to handle large-scale recommendation datasets and model training.

Model Configuration. We adopt a three-layer residual quantization structure with a codebook size of 256 per layer to provide a balanced trade-off between expressiveness and stability in residual quantization. The loss function balancing parameter λ is set to 0.1 after hyperparameter tuning. For the language model, we use Flan-T5-XL [1], which outputs 2048-dimensional representations that are then processed through our adapter network, building on the

Table 1: Overall performance comparison on three benchmark datasets. The best results are in bold, and the second-best are underlined. 'Improv.' denotes the relative improvement of HiST over the strongest baseline.

Dataset	Metric	Baseline Methods							Our Approach	
		P5	LightGCN	Caser	GRU4Rec	Bert4Rec	SASRec	TIGER	HiST	Improv.
Beauty	Recall@5	0.0163	0.0228	0.0205	0.0164	0.0203	0.0368	<u>0.0371</u>	**0.0434**	+16.98%
	Recall@10	0.0254	0.0421	0.0347	0.0283	0.0347	0.0510	<u>0.0576</u>	**0.0645**	+11.98%
	NDCG@5	0.0107	0.0136	0.0131	0.0099	0.0124	0.0239	<u>0.0244</u>	**0.0276**	+13.11%
	NDCG@10	0.0136	0.0198	0.0176	0.0137	0.0137	0.0323	<u>0.0326</u>	**0.0359**	+10.12%
Toys	Recall@5	0.0070	0.0165	0.0166	0.0097	0.0116	<u>0.0397</u>	0.0341	**0.0406**	+2.27%
	Recall@10	0.0121	0.0278	0.0270	0.0176	0.0203	<u>0.0565</u>	0.0512	**0.0597**	+5.66%
	NDCG@5	0.0050	0.0098	0.0107	0.0059	0.0071	<u>0.0275</u>	0.0227	**0.0285**	+3.64%
	NDCG@10	0.0066	0.0132	0.0141	0.0084	0.0099	<u>0.0345</u>	0.0289	**0.0362**	+4.93%
Sports	Recall@5	0.0061	0.0098	0.0116	0.0129	0.0115	0.0186	<u>0.0205</u>	**0.0248**	+20.98%
	Recall@10	0.0095	0.0184	0.0194	0.0204	0.0191	0.0254	<u>0.0312</u>	**0.0385**	+23.40%
	NDCG@5	0.0041	0.0061	0.0072	0.0086	0.0075	0.0120	<u>0.0132</u>	**0.0159**	+20.45%
	NDCG@10	0.0052	0.0087	0.0097	0.0110	0.0099	0.0149	<u>0.0166</u>	**0.0203**	+22.29%

Table 2: Dataset statistics of the evaluation benchmarks. "AvgLen" represents the average length of item sequences.

Dataset	#Users	#Items	#Interactions	AvgLen
Beauty	22,363	12,101	198,502	8.88
Toys	19,412	11,924	167,597	8.63
Sports	35,598	18,357	296,337	8.32

T5 text-to-text formulation [18]. For large-language-model-based hierarchical information extraction, we use DeepSeek-V3.1 (based on the DeepSeek-V3 technical report) [15]. The RQVAE employs a three-layer DNN structure with dimensions 768-256-128, producing final 64-dimensional item embeddings, following residual vector quantization principles [29]. This hierarchical reduction preserves semantic information while enabling efficient discrete representation. The adapter network transforms 2048-dimensional language model outputs to 128-dimensional embeddings suitable for integration with semantic IDs. This dimension alignment ensures compatibility between textual descriptions and discrete tokenization components.

Training Configuration. We use Adagrad optimizer with learning rate 0.001 and zero weight decay to maintain consistency with successful baseline configurations. For the main generative recommendation model, we apply a warmup-cosine learning rate scheduler with 1000 warmup steps, minimum ratio 0.01, and total scheduler steps matching the maximum training steps for stable convergence. For the RQVAE tokenizer, we do not use any learning rate scheduler and keep the Adagrad learning rate fixed throughout training.

5.3 Main Results

Table 1 presents the performance comparison across traditional and generative recommendation methods. Our HiST approach consistently outperforms existing baselines across all datasets and metrics.

Method Category Analysis. The results demonstrate clear performance hierarchies across different method categories. Among traditional sequential models (GRU4Rec, Caser, LightGCN), LightGCN achieves the strongest performance on Beauty dataset with 2.28% Recall@5. Transformer-based models (SASRec, BERT4Rec) show significant improvements, with SASRec reaching 3.68% Recall@5 on Beauty dataset. Generative recommendation models including TIGER and our proposed HiST consistently outperform all other categories, validating the effectiveness of treating recommendation as a sequence generation problem. This performance gap is particularly pronounced across all datasets, where generative methods show substantial improvements in both recall and NDCG metrics.

HiST Performance Analysis. Our proposed HiST method achieves the best performance across all datasets and metrics, demonstrating consistent improvements over the strong TIGER baseline. On the Beauty dataset, HiST achieves 4.34% Recall@5 compared to TIGER's 3.71%, representing a 16.98% relative improvement. The improvements are even more significant on the Sports dataset, where HiST reaches 2.48% Recall@5 versus TIGER's 2.05%, showing a 20.98% relative improvement. Using the same textual information, the naive feature-engineering baseline TIGER+Description does not always outperform TIGER, whereas TIGER+HiST still yields significant gains, indicating that the benefit comes from HiST's hierarchical semantic modeling rather than simply feeding more text (see the detailed analysis in Appendix 5). These results collectively validate the effectiveness of our hierarchical semantic tree augmentation in addressing the semantic branch drift problem and enhancing recommendation accuracy.

Cross-Dataset Consistency. The consistent performance improvements across the three Amazon domains (Beauty, Toys, Sports) suggest that our approach generalizes reasonably well across these benchmarks. The varying magnitudes of improvement across datasets reflect different levels of semantic complexity and the extent of semantic branch drift in each domain. The Sports dataset shows the most substantial relative improvements (20.98%), while the Toys

Table 3: Evaluation of HiST across different recommendation models on the Beauty dataset.

Model	Recall@5	NDCG@5	Improv.
RQVAE Tokenizer			
Baseline	0.0371	0.0244	-
Baseline + HiST	**0.0434**	**0.0276**	+16.98%
R-KMeans Tokenizer			
Baseline	0.0404	0.0268	-
Baseline + HiST	**0.0421**	**0.0297**	+12.9%
LC-Rec			
Baseline	0.0402	0.0257	-
Baseline + HiST	**0.0427**	**0.0260**	+14.3%
COBRA			
Baseline	0.0421	0.0268	-
Baseline + HiST	**0.0445**	**0.0305**	+7.3%

dataset shows more modest improvements (2.27%), suggesting that HiST is particularly effective in domains with complex hierarchical product structures.

5.4 Generalizability Analysis

An attractive property of HiST is that it can be plugged into different tokenization methods and recommendation architectures. To study this empirical generalization, we evaluate HiST's effectiveness when integrated with various tokenizers and recommendation models. Table 3 shows the performance improvements across different baseline models on the Beauty dataset.

Cross-Tokenizer Results. HiST achieves stable improvements across the tested tokenization approaches. When applied to RQVAE tokenizer, HiST achieves substantial improvements (+16.98%), effectively mitigating the severe semantic branch drift inherent in RQVAE's reconstruction-focused optimization. For R-KMeans tokenizer, HiST still provides meaningful improvements (+12.9%), albeit with smaller magnitude due to R-KMeans' naturally better semantic coherence within clusters. This behavior is expected: because R-KMeans already imposes K-means clustering in the encoder space, the additional LLM-induced semantic clustering in HiST is partially redundant and can even compete with the existing geometry. In contrast, vanilla RQVAE does not enforce such explicit clustering and exhibits stronger branch drift, so the hierarchical summaries injected by HiST provide a more complementary signal and lead to larger relative gains. The gap between the RQVAE and R-KMeans results reflects the varying degrees of semantic branch drift across tokenization methods, validating HiST's adaptive enhancement capability.

Cross-Architecture Results. Beyond tokenizer compatibility, HiST yields consistent gains across the tested recommendation architectures. The results with LC-Rec (+14.3%) and COBRA (+7.3%) demonstrate HiST's versatility across different recommendation paradigms. The consistent improvements validate key aspects of HiST's design: architecture-agnostic semantic tree construction, adapter-based integration mechanism, and dual-component loss design. The varying improvement magnitudes provide insights into HiST's effectiveness patterns, where methods with more severe semantic branch drift benefit more substantially from hierarchical augmentation.

Table 4: Basic ablation study on the Beauty dataset. Each component contributes to the final performance.

Method	Recall@5	NDCG@5	Improv.
Baseline	0.0371	0.0244	-
HiST (Full)	**0.0434**	**0.0276**	+16.98%
w/o Layer-1	0.0429	0.0272	+15.6%
w/o Layer-2	0.0398	0.0255	+7.3%
w/o Layer-3	0.0412	0.0263	+11.1%
w/o Adapter	0.0342	0.0228	-7.8%

5.5 Ablation Study

We conduct ablation studies on different components of HiST to determine their individual contributions. Table 4 shows the basic results.

The ablation analysis reveals the hierarchical importance of different semantic layers in HiST. Removing Layer-2 causes the most significant performance degradation among layers, dropping from 16.98% to 7.3% improvement, confirming its critical role in capturing semantic subclass information that bridges high-level categories and fine-grained item details. Layer-3 removal results in moderate performance drop to 11.1% improvement, demonstrating the value of fine-grained semantic details for precise item characterization. Layer-1 shows the smallest individual impact with only slight degradation to 15.6%, suggesting that while it provides categorical foundations, its contribution is most valuable when combined with other layers.

The adapter mechanism plays an important role in HiST's effectiveness. Removing the adapter network leads to a substantial performance degradation, dropping below the baseline with -7.8% performance loss. This result suggests that without proper dimensional alignment between text embeddings and semantic ID embeddings, the hierarchical semantic enhancement is difficult to leverage and can even hurt model performance.

6 Conclusion

In this paper, we identify semantic branch drift as a critical limitation of tokenization in generative recommendation, where semantically similar items are assigned to different branches, degrading model understanding and representation quality. To address this issue, we propose HiST (Hierarchical Semantic Tree Augmentation), a general framework that introduces hierarchical semantic information to mitigate this gap. HiST constructs a semantic tree from a trained tokenizer, derives hierarchical descriptions via bottom-up refinement process, and integrates them with semantic IDs to form enhanced representations. Through dual-component training objectives, HiST jointly models semantic ID prediction and hierarchical description learning, enabling effective modeling of both discrete structure and semantic relationships. Experiments show that HiST consistently improves recommendation performance across datasets, tokenization methods, and architectures.

Acknowledgments

This work was supported by Brain Science and Brain-like Intelligence Technology—National Science and Technology Major Project under Grant 2022ZD0208700, and National Natural Science Foundation of China under Grant 62376264.

References

[1] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.

[2] Hao Ding, Yifei Ma, Anoop Deoras, Yuyang Wang, and Hao Wang. 2021. Zero-shot recommender systems. *arXiv preprint arXiv:2105.08318* (2021).

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The faiss library. *IEEE Transactions on Big Data* (2025).

[4] Dengzhao Fang, Jingtong Gao, Chengcheng Zhu, Yu Li, Xiangyu Zhao, and Yi Chang. 2025. Hid-vae: Interpretable generative recommendation via hierarchical and disentangled semantic ids. *arXiv preprint arXiv:2508.04618* (2025).

[5] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.

[6] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.

[7] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).

[8] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating long semantic IDs in parallel for recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 956–966.

[9] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 585–593.

[10] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvesh Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6420–6425.

[11] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[12] Hyeyoung Ko, Suyeon Lee, Yoonseo Park, and Anna Choi. 2022. A survey of recommendation systems: recommendation models, techniques, and application fields. *Electronics* 11, 1 (2022), 141.

[13] Zhirui Kuai, Zuxu Chen, Huimu Wang, Mingming Li, Dadong Miao, Wang Binbin, Xusong Chen, Li Kuang, Yuxing Han, Jiaxing Wang, et al. 2024. Breaking the hourglass phenomenon of residual quantization: Enhancing the upper bound of generative retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 677–685.

[14] Jiacheng Li, Ming Wang, Jin Li, Jinmiao Fu, Xin Shen, Jingbo Shang, and Julian McAuley. 2023. Text is all you need: Learning language representations for sequential recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1258–1267.

[15] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[16] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.

[17] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[18] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[19] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[20] Teng Shi, Jun Xu, Xiao Zhang, Xiaoxue Zang, Kai Zheng, Yang Song, and Enyun Yu. 2025. GenSAR: Unifying Balanced Search and Recommendation with Generative Retrieval. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 124–134.

[21] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative retrieval with semantic tree-structured identifiers and contrastive learning. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 154–163.

[22] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[23] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.

[24] Yifan Wang, Weinan Gan, Longtao Xiao, Jieming Zhu, Heng Chang, Haozhao Wang, Rui Zhang, Zhenhua Dong, Ruiming Tang, and Ruixuan Li. 2025. Act-With-Think: Chunk Auto-Regressive Modeling for Generative Recommendation. *arXiv preprint arXiv:2506.23643* (2025).

[25] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.

[26] Longtao Xiao, Haozhao Wang, Cheng Wang, Linfei Ji, Yifan Wang, Jieming Zhu, Zhenhua Dong, Rui Zhang, and Ruixuan Li. 2025. Unger: Generative recommendation with a unified code via semantic and collaborative integration. *ACM Transactions on Information Systems* 44, 2 (2025), 1–31.

[27] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, et al. 2025. Sparse meets dense: Unified generative recommendations with cascaded sparse-dense representations. *arXiv preprint arXiv:2503.02453* (2025).

[28] Hongzhi Yin, Liang Qu, Tong Chen, Wei Yuan, Ruiqi Zheng, Jing Long, Xin Xia, Yuhui Shi, and Chengqi Zhang. 2025. On-device recommender systems: A comprehensive survey. *Data Science and Engineering* (2025), 1–30.

[29] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2021), 495–507.

[30] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, Xiaofang Zhou, et al. 2019. Feature-level deeper self-attention network for sequential recommendation.. In *IJCAI*. 4320–4326.

[31] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.

A Reproducibility and Code Availability

To facilitate reproducibility, we provide comprehensive implementation details and supporting resources across the appendix: Appendix A presents the anonymous implementation and usage instructions, Appendix B/C detail the training configurations and hyperparameters, and Appendix G provides the prompt design and templates for the offline LLM component.

The codebase is available at https://github.com/Penation/HiST. The repository supports the full HiST pipeline, including data preparation, embedding generation, semantic ID training and generation, hierarchical semantic tree construction, generative recommendation model training, and final recommendation generation. Each component is accompanied by detailed documentation and command-line examples to support reproducibility and practical usage. Due to copyright restrictions, we are unable to release the processed datasets. However, the repository provides a complete data preprocessing pipeline and tutorials, enabling users to reproduce the exact data splits starting from the original Amazon datasets.

B Tokenizer, Semantic Tree, and Adapter Implementation Details

In this appendix we provide additional implementation details of the tokenizer training pipeline, semantic tree construction, and the adapter network used to fuse hierarchical descriptions with semantic IDs.

B.1 RQVAE Tokenizer Training

We adopt a residual vector quantization (RQVAE) tokenizer with $L = 3$ quantization layers and $K = 256$ codewords per layer. The encoder that produces continuous item embeddings is a three-layer MLP with hidden sizes $768 \to 256 \to 128$, followed by a linear projection to a 64-dimensional latent space that is fed into the residual quantization module. Each codebook is trained to minimize a weighted sum of quantization loss and reconstruction loss following [29], and the residual between the input and the reconstructed embedding is successively quantized by deeper layers. All RQVAE models are trained with Adagrad (learning rate 0.001, weight decay 0.0) without any learning rate scheduler.

For each dataset, we train the RQVAE tokenizer on a single NVIDIA RTX 5090 GPU using mixed-precision training. With the above configuration and codebook size, training converges within approximately 15 minutes per dataset (Amazon Beauty, Toys, and Sports), which is negligible compared to the end-to-end recommendation model training pipeline.

B.2 Semantic Tree Construction Pipeline

After training the tokenizer, we construct the hierarchical semantic tree entirely offline. The pipeline consists of three stages:

(1) **Semantic ID extraction.** We first run batched inference of the trained RQVAE over the full item catalog to obtain a three-layer semantic identifier $z(x) = (z_1, z_2, z_3)$ for each item, with $z_\ell \in \{1, \ldots, 256\}$.

(2) **Group-level description generation.** Items that share the same (z_1, z_2, z_3) are grouped together as leaf nodes of the tree. For each such group we aggregate the original product titles and descriptions into a prompt and query a large language model to obtain a concise natural-language summary capturing the common semantics of that semantic cluster (see Appendix C for prompt design).

(3) **Embedding and semantic clustering.** We encode the generated summaries using the Flan-T5-XL encoder [1] implemented with Hugging Face Transformers, obtaining 2048-dimensional embeddings. On the intermediate level (e.g., groups formed by (z_1, z_2) or $z_{1:2}$), these summary embeddings are further clustered with MiniBatchKMeans [19] using up to 256 centroids. Each centroid corresponds to a higher-level semantic cluster that merges multiple fine-grained leaf groups, and serves as an internal node in the tree. Coarser nodes (such as layer-1) are then obtained by aggregating and summarizing the corresponding cluster-level descriptions, yielding a three-level semantic structure built in a bottom-up, data-driven manner.

B.3 Adapter Network Architecture

The adapter network $\mathcal{A}(\cdot)$ maps 2048-dimensional description vectors to the Transformer hidden space used for semantic IDs. In our implementation, $\mathcal{A}(\cdot)$ is a lightweight two-layer feed-forward network with non-linear activation and dropout:

$$\begin{aligned} h_1 &= \mathrm{GELU}(W_1 e_{\mathrm{desc}} + b_1), \\ h_2 &= W_2 h_1 + b_2, \\ d &= h_2, \end{aligned} \tag{9}$$

where $e_{\mathrm{desc}} \in \mathbb{R}^{2048}$ is the input description vector, $W_1 \in \mathbb{R}^{512\times 2048}$, $W_2 \in \mathbb{R}^{\mathrm{emb}\times 512}$, and we insert a dropout layer with rate 0.1 after the GELU activation. The same adapter is shared across hierarchy levels.

The adapted description embedding d is then combined with the original SID embedding e for each hierarchy position. Concretely, we concatenate $[e, d]$ and pass it through a small fusion block consisting of a linear projection back to the Transformer hidden size followed by LayerNorm. The output of this fusion block is the HSID-aware token embedding that is fed into the encoder. During training, both the adapter and fusion parameters are learned jointly with the generative recommender under the dual loss $\mathcal{L} = \mathcal{L}_{\mathrm{SID}} + \lambda \mathcal{L}_{\mathrm{desc}}$, where $\mathcal{L}_{\mathrm{desc}}$ is a contrastive loss defined over semantic description vectors.

C LLM Usage and Training Infrastructure

C.1 LLM-based Hierarchical Description Extraction

For hierarchical description extraction we rely on two types of language models. First, we use DeepSeek-V3.1 [15] via its official HTTP API to generate textual summaries for item groups at different hierarchy levels. Our client issues JSON-based requests with temperature set to 0.3 and a maximum generation length of 1000 tokens, while keeping other decoding parameters (such as top-p and penalties) at their API defaults. The requests include explicit instructions to produce concise, non-redundant summaries tailored to each semantic branch and are batched and retried with exponential backoff to respect rate limits and improve robustness.

Items ID Sequence: [620]-[459]-[621]-[418]-[168]

RQVAE SID Sequence: [10_51_91]-[52_140_40]-[149_68_104]-[174_147_247]-[242_45_207]

R-Kmeans SID Sequence: [200_251_135]-[56_85_185]-[238_**122**_138]-[172_**122**_138]-[148_**122**_22]

[620] Title: Urban Decay Eyeshadow Primer Potion (10 ml); Brand: Unknown; Categories: ['Beauty', 'Makeup', 'Eyes', 'Concealer & Base']; Price: Unknown;

[459] Title: Avalon Biotin B-Complex Thickening Conditioner, 14 Ounce; Brand: Avalon Organics; Categories: ['Beauty', 'Hair Care', 'Conditioners']; Price: $9.49;

[621] Title: Philosophy Take A Deep Breath Oil-free Energizing Oxygen Gel Cream Moisturizer, 2 Ounce; **Brand: Philosophy;** Categories: ['Beauty', 'Skin Care', 'Face', 'Creams & Moisturizers', 'Fluids & Lotions', 'Fluids']; Price: $25.0;

[418] Title: Philosophy Coconut Frosting Shampoo/Shower Gel/Bubble Bath, 16 Ounces; **Brand: Philosophy;** Categories: ['Beauty', 'Bath & Body', 'Cleansers', 'Body Washes']; Price: $14.0;

[168] Title: Philosophy Purity Made Simple One-Step Facial Cleanser, 16 Ounces; **Brand: Philosophy;** Categories: ['Beauty', 'Skin Care', 'Face', 'Cleansers', 'Washes']; Price: $31.0;

Figure A.1: Detailed case study showing semantic ID assignments for a user purchase sequence, highlighting the difference in brand consistency between RQVAE and R-KMeans tokenizers.

Second, we use the publicly available Flan-T5-XL encoder from Hugging Face to transform these summaries into dense embeddings, as detailed in the SummaryEncoder utility of our codebase. The model is loaded locally on GPU, and all parameters are frozen so that the encoder serves purely as a feature extractor during semantic tree construction and subsequent analysis.

The overall runtime of semantic tree construction is dominated by the external DeepSeek API calls. On the Amazon Beauty dataset, with moderate parallelism, generating all group-level summaries typically takes a few hours wall-clock time, while the subsequent local encoding and clustering steps complete within tens of minutes on a single RTX 5090 GPU and commodity CPUs.

C.2 Training Infrastructure and Runtime

All experiments are conducted on a server equipped with 8 NVIDIA RTX 5090 GPUs. Unless otherwise specified, we train the generative recommendation Transformer with HSID representations using 8 GPUs in data-parallel fashion, a global batch size chosen to fully utilize GPU memory, and the Adagrad optimizer with learning rate 0.001 and cosine learning-rate decay. For each dataset, end-to-end training of the Transformer backbone together with the HSID embeddings and adapters converges within roughly 2 hours on 8 RTX 5090 GPUs.

As summarized above, the total offline cost per dataset is thus dominated by three components: (1) RQVAE tokenizer training on a single RTX 5090 (about 15 minutes), (2) semantic tree construction time determined primarily by the DeepSeek-V3.1 API throughput, and (3) Transformer-based generative recommender training on 8 RTX 5090 GPUs (about 2 hours).

D Brand Consistency Analysis

Categories and brands are equally important for item description in e-commerce recommendation systems, with brands serving as a crucial complement to categories in understanding product semantics. To further demonstrate the semantic branch drift phenomenon, we provide a detailed case study examining the relationship between brand information and layer-2 semantic IDs. This analysis focuses on brand consistency as it represents a fundamental aspect of item similarity that should be preserved in hierarchical tokenization. We analyze a representative user sequence from the Amazon Beauty dataset to illustrate how different tokenizers handle brand consistency across hierarchical levels and examine the implications for recommendation model understanding.

We analyze a representative user sequence containing items from the Philosophy brand (items 621, 418, 168), which should ideally be grouped together in the semantic hierarchy. As illustrated in Figure A.1, the analysis reveals significant differences between tokenizers in maintaining brand consistency. In RQVAE's encoding, the three Philosophy products receive completely different layer-2 IDs (68, 147, 45), demonstrating severe semantic branch drift where items from the same brand are scattered across different branches of the hierarchical tree. This fragmentation disrupts the natural semantic organization that users and recommendation systems expect. In contrast, R-KMeans shows better semantic coherence with all three items receiving the same layer-2 ID of 122, preserving brand-level similarity in the tokenization process. This consistency enables recommendation models to better understand brand relationships and maintain semantic continuity throughout user interaction sequences, while RQVAE's scattered assignments prevent effective learning of brand-specific patterns and user brand preferences, directly impacting the model's ability to understand user preferences and generate meaningful recommendations that align with brand loyalty patterns.

E Baseline Methods

We compare our proposed HiST with both traditional and generative recommendation methods:

- **LightGCN** captures high-order collaborative signals via lightweight graph convolution without feature transformations and nonlinear activations, focusing on neighborhood aggregation.
- **Caser** leverages convolutional filters to extract both point-level and union-level sequential patterns from user behavior sequences.
- **GRU4Rec** utilizes Recurrent Neural Networks to capture temporal dependencies in user behavior sequences through gated recurrent units.
- **P5** transforms various recommendation tasks into natural language sequences, providing a universal framework for recommendation completion through text-to-text generation.
- **BERT4Rec** adapts bidirectional encoder representations from transformers for sequential recommendation by modeling user behavior sequences with masked language model objectives.
- **SASRec** employs self-attention mechanisms to model long-range dependencies and capture dynamic user preferences in sequential interactions.

- **TIGER** introduces semantic IDs via residual quantization to represent items as discrete token sequences, enabling generative retrieval through autoregressive decoding.
- **LC-Rec** leverages RQ-VAE-based identifiers but focuses on aligning language and collaborative semantics for improved semantic consistency.
- **COBRA** integrates cascaded sparse and dense representations through a coarse-to-fine generation process.

F TIGER Implementation and Fairness Discussion

F.1 Implementation Choice

The original TIGER implementation is not publicly released. In our work, we therefore rely on the open-source GRID framework [10], which provides a faithful reimplementation of TIGER and a unified pipeline for generative recommendation with semantic IDs. We follow GRID's public configuration on the Amazon 5-core datasets, including user/item filtering, chronological sorting, leave-one-out splitting, and full-ranking evaluation with Recall@K and NDCG@K. All models in our study (TIGER, HiST, and other baselines) are trained and evaluated under the same data preprocessing and evaluation pipeline, ensuring that the *relative* improvements reported for HiST over TIGER are fair and reliable, even if the absolute numbers may differ from those in the original TIGER paper due to implementation and preprocessing differences.

Table 5: Effect of hierarchical semantic augmentation versus naive description injection on the Beauty dataset (RQ-VAE tokenizer). All methods share the same implementation pipeline from GRID.

Method	Recall@5	NDCG@5
TIGER	0.0371	0.0244
TIGER + Description	0.0362	0.0231
TIGER + HiST	**0.0434**	**0.0276**

F.2 Effect of Additional Textual Information

HiST introduces additional information compared with the original TIGER: beyond semantic IDs, it explicitly leverages item-level textual metadata (titles, categories, descriptions) through LLM-based hierarchical summaries. This design reflects a realistic industrial setting, where such metadata is almost always available, and our goal is to study how to systematically exploit these signals in SID-based generative retrieval.

To address the fairness concern that HiST might simply benefit from using more information, we construct an additional baseline, *TIGER + Description*, where we encode the raw item descriptions using the same language model and adapter architecture as in HiST, but *without* building a semantic tree. The resulting description embeddings are directly fused with semantic ID embeddings in the TIGER encoder/decoder. Table 5 summarizes the comparison on the Beauty dataset.

In our experiments, naively injecting raw description embeddings into TIGER (*TIGER + Description*) does not outperform the original TIGER and can even slightly hurt performance on some metrics, likely because the additional free-form text introduces noisy or redundant signals that are not aligned with the residual-quantized code space. By contrast, TIGER + HiST consistently and significantly outperforms TIGER. This indicates that HiST's gains mainly stem from its *hierarchical semantic tree construction and structured integration mechanism*, rather than simply feeding more textual features into the model.

F.3 Effect of Different LLM Backends

Beyond the choice of tokenizer, HiST is also agnostic to the specific large language model used for hierarchical description extraction. In practice, our implementation allows replacing DeepSeek-V3.1 with alternative chat models such as Qwen2.5-Chat or GPT-5.1 by simply changing the API endpoint and model name while keeping prompts and decoding settings fixed. Once such experiments are conducted, the results can be summarized as in Table 6 to analyze the sensitivity of HiST to different LLM backends.

Table 6: Effect of different LLM backends for hierarchical description extraction on the Beauty dataset (RQVAE tokenizer). All methods share the same implementation pipeline from GRID.

Method	Recall@5	NDCG@5
TIGER (no LLM)	0.0371	0.0244
TIGER + HiST (DeepSeek-V3.1)	0.0415	0.0268
TIGER + HiST (Qwen2.5-Chat)	0.0434	0.0276
TIGER + HiST (GPT-5.1)	0.0428	0.0270

We expect that, as long as the LLM can produce coherent branch-level summaries, different backends will lead to broadly similar improvements over TIGER, since the language model is only used to construct the offline semantic tree and does not participate in online inference. A detailed empirical comparison of multiple LLM providers is left as future work and can be reported using the structure illustrated above.